

Python程序设计复习

考试信息

- 考试时间和地点

- 2026年6月21日（周日上午） 08:00-09:30 闭卷（纸质，没有计算机调试）考试
- 地点：H3206

- 考试题型：

1. 阅读程序写输出3道（30%）。 $3 * 10 = 30\%$

2. 程序填空题1道（10%）。 2处空， $2 * 5 = 10\%$

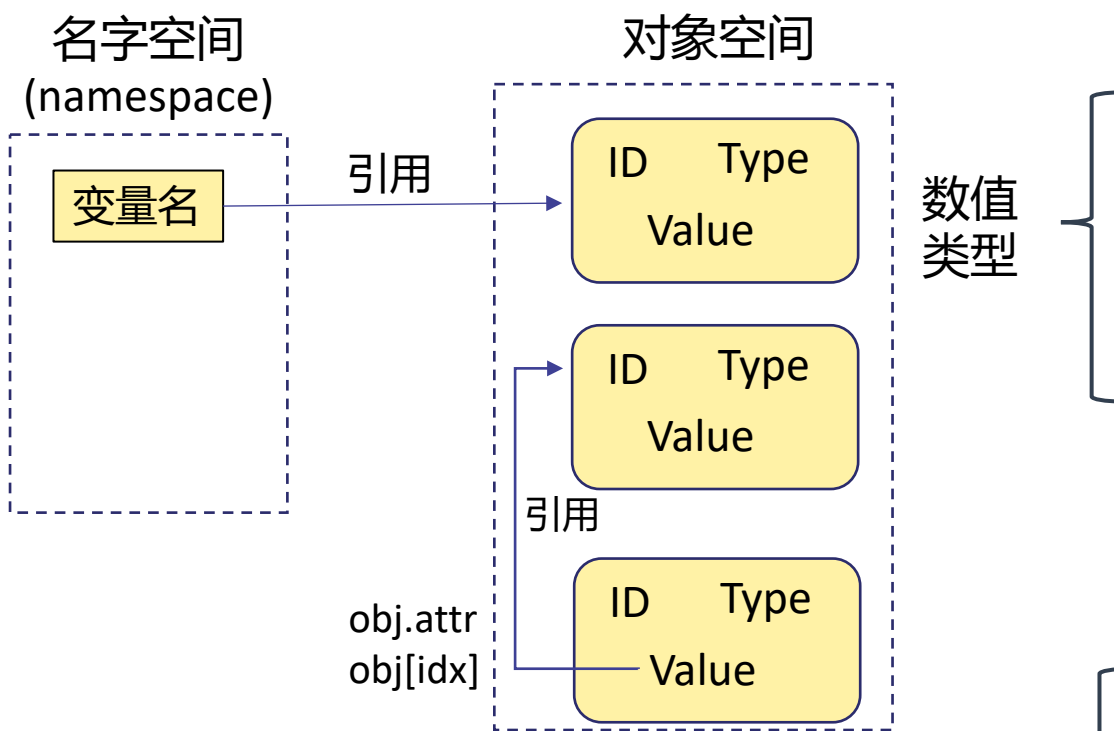
3. 程序改错题2道（20%）。 每题2处错误， $2 * 2 * 5 = 20\%$ 指出错误行的位置并且改正

4. 编程题3道（40%）。 $10 + 10 + 20 = 40\%$

主要内容

- 基础知识：数据类型、赋值语句、运算符、内置函数、模块
- 程序结构：函数、分支结构、循环结构、异常处理
- 数据结构：
 - 字符串
 - 列表
 - 元组
 - 字典
 - 集合
- 文件
- 正则表达式

Python程序设计概览：数据类型



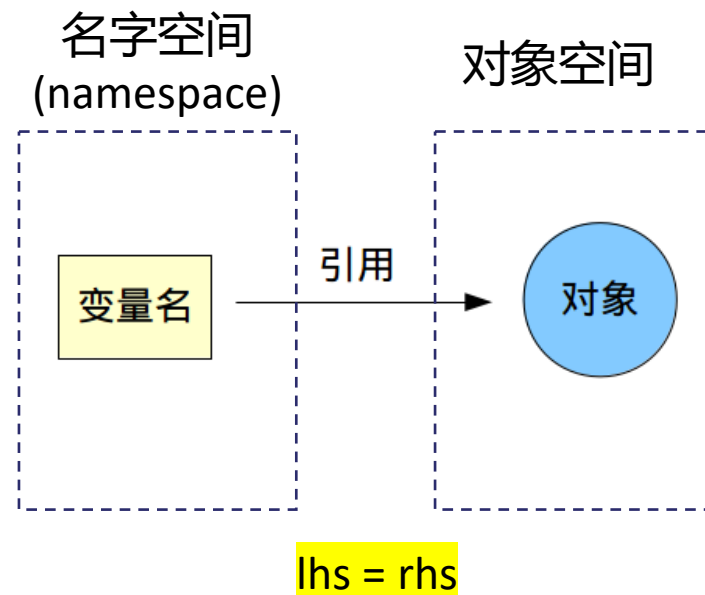
id(object)	返回object的ID
type(object)	返回object的类型
isinstance(obj, class or tuple)	obj是否是某个或者某些类型(包括其各级子类型的实例)

- 可变与不可变：对象创建后，是否可以改变对象的值
 - 可变对象：list, dict, set

类型	描述	示例
int	整数	42, 20493, 0
float	实数(浮点数)	7.35, 4., 6.022e23
complex	复数	1+2j, 3.4-1j
bool	布尔	True, False
NoneType	空类型	None
str	字符串	"Hello.", 'abc 1 2', ''
range	等差数列整数	range(10) range(0, 10, 2)
list	列表	[1, 2, 3]
tuple	元组	(1, 2, 3)
dict	字典	{'name': 'tony', 'age': 22}
set	集合	{1, 2, 3}
function	函数对象	def语句和lambda表达式
class	类对象 (类型)	class语句
module	模块对象	import语句

基础知识：对象模型

- Python对象：任何数据都是对象
 - ID: 该对象的唯一标识，内部表示方式，一般不关心其具体取值
 - Type: 对象的类型，决定了其取值范围以及支持的运算
 - Value: 对象的取值，根据值是否可以修改分为不可变和可变对象
- (变量) 名字：表示对于某个对象的引用 (reference)
 - name与object之间的绑定一般通过**赋值语句**来完成
 - name可以属于不同的命名空间namespace (模块、函数)
 - 所谓name的类型实际上指的是name所指向的对象的类型：无需预先声明name的类型
 - **名字出现在RHS表示引用该名字指向对象的值**
 - **名字出现在LHS表示将RHS计算后得到的对象与名字绑定起来**
 - 解释器会自动回收那些不再有名字绑定的对象
 - 可以通过del 语句解除名字与对象的绑定



- 掌握对象与名字的概念
- 了解对象的三个基本属性
- 掌握isinstance(obj, class or tuple)

赋值

赋值类语句

- 赋值语句: $LHS = RHS$, 首先计算rhs, 然后赋值给LHS
 - 变量名: $a = 4 + 5$
 - 属性: $obj.attr = 4$
 - 下标: $s[0] = 4$, $d['key'] = 4$
 - 切片: $s[0:4] = iterable$
 - 多个对象引用 (序列解包): $x, y = 1, 2$; $x, y, *z = range(10)$;
 $idx, (key, value) = enumerate(d.items(), 1)$
- 链式赋值: $a = b = c = expr$, 右结合
- 复合赋值: $+=$ $-=$ $*=$ $\%=$ $//=$ $/=$ 等
 - $LHS\ op = RHS$, LHS为不可变对象时, 等价于 $LHS = LHS\ op\ RHS$, op 为算术运算符
 - (不要求) LHS为列表对象时, 原地附加或重复: $listobj\ +=\ iterable$; $listobj\ *=\ iterable$
- (不要求) 海象运算符: `(text := input('...')).lower() in ('quit', 'exit')`
- import语句: `import math; from math import sin, cos`
- def语句
- 参数传递
- for循环中的for变量

名字空间
(namespace)

变量名

引用

对象空间

ID Type
Value

ID Type
Value

引用

ID Type
Value

obj.attr
obj[idx]

del 语句解除名字与对象的绑定

- del a
- del s[idx], del s[slice]
- del d[key]

基础知识：合法的变量名

什么是一个合法的变量名？

什么是一个合法的变量名？

- 必须以**字母或下划线开头**，其后的字符可以是**字母、下划线或数字**
- 以下划线开头的变量在Python中有特殊含义
- 变量名中不能有空格以及标点符号（括号、引号、逗号、斜线、反斜线、冒号、句号、问号等）
- 英文字母的**大小写敏感**，例如student和Student是不同的变量
- 一些特殊的标识符保留为Python关键字，不能用作变量名

Python关键字: 共35个

关键字	含义
None	常量, 空值, 首字母必须大写
True	常量, 逻辑真, 首字母必须大写
False	常量, 逻辑假, 首字母必须大写
from	明确指定从哪个模块中导入什么对象
import	导入模块或模块中的对象
as	在import或except语句中给对象起别名
assert	断言语句, 确认某个条件必须满足
for	循环结构, 列表推导式等
while	循环结构
continue	循环结构中提前结束本次循环
break	循环结构中提前结束break所在层次的循环
if	用在选择结构中, 检查条件是否成立。列表推导式等
elif	用在选择结构中, 表示else if的意思
else	用在选择结构、循环结构和异常处理结构中
except	异常处理, 捕获特定类型的异常
finally	异常处理, 不论是否发生异常都会执行的代码
del	删除 (解绑) 对象或对象成员

关键字	含义
def	定义函数
class	定义类
global	声明全局变量
in	成员测试
is	同一性测试
lambda	定义lambda表达式, 匿名函数
nonlocal	声明外层函数的变量
not	逻辑非运算
and	逻辑与运算
or	逻辑或运算
pass	空语句, 执行该语句时什么都不做
raise	显式抛出异常
return	在函数中用来返回值
try	异常处理, 限定可能会引发异常的代码块
with	上下文管理, 具有自动管理资源的功能
yield	在生成器函数中用来生成值
async	声明一个函数为异步函数
await	声明程序挂起

python3.10引入match语句
基于值和结构进行匹配
match subject:
 case pattern1:
 # 代码块1
 case pattern2:
 # 代码块2
 case _:
 # 默认分支 (可选)

基础知识：数值类型、算术运算符

- 数值类型：整数(int)、浮点数(float)、复数(complex, 不要求)、布尔 (bool)
 - 整数字面量：十进制整数、十六进制、八进制和二进制整数 (0x/0o/0b...) **只需要掌握十进制整数**
 - 浮点数字面量：
 - 小数表示：3.14 10. .001
 - **科学计数法 (整数或小数表示 + e或E + 整数)** : 15e-2 ($=15*10^{-2}=0.15$) 3.14e-10 1.0E100
 - **/: 浮点除法或真除法, 结果为浮点数**
 - **//: 整除法, 如果都为整数, 则结果为整数, 如果其中有浮点数则结果为浮点数**
 - **求模运算: 可用于判断是否整除, 判断奇偶性, 可与//结合在一起获得整数的各位数字**
 - 幂运算: **
- 算术运算: + - * / // % 一元+或- **
 - 优先级顺序: **最高, 接下来为一元改符号, 然后是乘除, 最后为加减
 - 结合: 之后**为右结合, 其余为左结合
 - **除法运算: / 为真除法, 最后为浮点数, // 为整除, 求整商, 全部为整数时结果为整数, 有浮点数则结果为浮点数;**
 - 除了/运算外, 两个不同数字类型之间的混合运算通过**类型提升**提升为更高的类型后计算, int→float→complex
- 内置数学函数: abs、divmod、max、min、sum、round等
- math模块: sqrt、log等
- dir和help函数查找有哪些方法以及方法的帮助

- 掌握真除法和求整商运算符的区别
- 掌握求整商和求余(求模) 运算符, 知道如何实际运用
- 掌握常用内置数学函数, 了解math模块的sqrt, log等函数

比较类和逻辑运算符

- 比较运算符： **所有比较类运算符的优先级相等**
 - 大小和相等比较： `==` `!=` `>` `<` `>=` `<=`
 - 成员判断： `in` `not in`
 - 同一性判断： `is` `is not`
 - 比较运算符支持连续比较，但是没有传递性 `x < y < z`, `x < y > z`
- 逻辑运算符： **`not` `>` `and` `>` `or` `>` `value1 if cond else value2`**
 - 短路逻辑：如果当前表达式的真值判断结果决定了最终的真值判断结果，后续的表达式不进行求解
 - `not expr`: 逻辑非, `True`或者`False`
 - `expr1 and expr2`: 逻辑与
 - 第一个真值判断为假的表达式或者最后一个表达式
 - 替代值: `prompt and f'prompt >>>' ==>` 空字符串或者 `prompt>>>`
 - `expr1 or expr2`: 逻辑或
 - 第一个真值判断为真的表达式或者最后一个表达式
 - 缺省值: `prompt or 'default' ==>` 非空字符串或者缺省的'`default`'
 - `value1 if cond else value2`: `value1`或`value2`其中之一

运算符优先级

逻辑运算

比较运算

算术运算

运算符	描述
a := expr	海象赋值运算符, python3.8
lambda parameter: expr	lambda表达式
value1 if cond else value2	二选一条件表达式
or	布尔 "或"
and	布尔 "与"
not x	布尔 "非"
in, not in	成员测试
is, is not	同一性测试
<, <=, >, >=, !=, ==	比较
	按位或; 集合并集
^	按位异或; 集合异或 (对称差)
&	按位与; 集合交集
<<, >>	左移和右移
+	加法; 列表、元组和字符串的连接
-	减法; 集合差集
*	乘法; 列表、元组和字符串的重复
@	矩阵乘法
/, //	真除法、整除 (原点方向取整)
%	取余; 字符串格式化

运算符	描述
+x, -x	正负号, 一元运算或单目运算
~x	按位求反
**	求幂
x.attribute	属性
x[index] x[index:index]	下标和切片
f(arguments...)	函数调用
(expression,...)	元组字面量, 生成器表达式
[expression,...]	列表字面量, 列表推导式
{key:datum,...}	字典字面量, 字典推导式
{expression, ...}	集合字面量, 集合推导式
'expression,...'	字符串字面量

- 优先级越低, 越最后计算
 - 除了**为右结合外, 其他运算符都是左结合
- 掌握运算符优先级以及结合方式
 - 先算后比, 比完之后进行逻辑运算
 - 一元优先级比两元优先级更高
 - and比or优先级更高

内置函数

函数		功能
构造函数	bool(object)	真值判断
	complex(x)	复数
	int(x)	整数
	float(x)	浮点数
	str(object)	字符串
	list(iterable)	列表
	tuple(iterable)	元组
	dict(iterable)	字典
	set(iterable)	集合
字符串相关	bytes(iterable_of_ints)	字节串
	repr(obj)	描述obj的字符串
	ascii(obj)	类似于repr, 只能为ASCII字符
	bin(n)	二进制整数字符串(0b...)
	oct(n)	八进制整数字符串(0o...)
	hex(n)	十六进制整数字符串(0x...)
	chr(i)	unicode码为i的字符串
	ord(ch)	该字符对应的unicode码
	format(value, spec)	对value进行格式化

函数		功能
数学函数	abs(x)	绝对值或复数x的模
	divmod(x, y)	(整商//, 余数%)
	max(..., key=None)	最大值
	min(..., key=None)	最小值
	sum(iterable)	求和
	pow(x, y)	x^y 幂函数 $x**y$
输入输出等	round(x[,n])	取整 (四舍六入, 五成双)
	input(prompt)	输入
	print(...)	输出
	eval(expr_str)	对字符串中的表达式求解后返回求解后的结果
对象模型	exec(stat_str)	执行字符串中的语句, 返回None
	id(object)	返回object的ID
	type(object)	返回object的类型
	isinstance(obj, class or tuple)	obj是否是某个或者某些类型(包括其各级子类型)的实例
	issubclass(cls, cls2)	cls是否为cls2的子类型
	help(), help(obj)	查看帮助
	dir([obj])	列出obj或当前名字空间名字
	globals()	查看全局名字空间
	locals()	查看当前名字空间
	vars([obj])	查看当前或者obj的名字空间

内置函数

函数		功能
构造函数	bool(object)	真值判断
	complex(x)	复数
	int(x)	整数
	float(x)	浮点数
	str(object)	字符串
	list(iterable)	列表
	tuple(iterable)	元组
	dict(iterable)	字典
	set(iterable)	集合
字符串相关	bytes(iterable_of_ints)	字节串
	repr(obj)	描述obj的字符串
	ascii(obj)	类似于repr，只能为ASCII字符
	bin(n)	二进制整数字符串(0b...)
	oct(n)	八进制整数字符串(0o...)
	hex(n)	十六进制整数字符串(0x...)
	chr(i)	unicode码为i的字符串
	ord(ch)	该字符对应的unicode码
	format(value, spec)	对value进行格式化

函数		功能
数学函数	abs(x)	绝对值或复数x的模
	divmod(x, y)	(整商//, 余数%)
	max(..., key=None)	最大值
	min(..., key=None)	最小值
	sum(iterable)	求和
	pow(x, y)	x^y 幂函数 $x**y$
输入输出等	round(x[,n])	取整 (四舍六入, 五成双)
	input(prompt)	输入
	print(...)	输出
	eval(expr_str)	对字符串中的表达式求解后返回求解后的结果
对象模型	exec(stat_str)	执行字符串中的语句, 返回None
	id(object)	返回object的ID
	type(object)	返回object的类型
	isinstance(obj, class or tuple)	obj是否是某个或者某些类型 (包括其各级子类型)的实例
	issubclass(cls, cls2)	cls是否为cls2的子类型
	help(), help(obj)	查看帮助
	dir([obj])	列出obj或当前名字空间名字
	globals()	查看全局名字空间
	locals()	查看当前名字空间
	vars([obj])	查看当前或者obj的名字空间

输入输出

input(prompt=None):

- 如果传递prompt, 则首先显示提示符。等待用户输入, 直到用户键入回车时input返回, 将**回车之前的内容(字符串)**返回给调用者
- 注意**返回的是str**, 可能需要**通过int/float等转换为数字类型的对象**, 如果输入多个数据时可通过split进行分割后再转换

```
text = input('....')
if not text:
    # 输入为空字符串, 结束
```

- 掌握input()函数
- 掌握 输入一个合法的整数或者浮点数, 和循环、异常处理一起结合处理
- 掌握输入多个整数: 输入一行文本, 将文本分割

```
while True:
    try:
        text = input('...')
        t = int(text)
        ...
        break
    except Exception as e:
        print(type(e), e) # 输入有误
```

如果要输入的是多个参数(整数), 以某个分隔符隔开, 比如/

```
text = text.strip()
args = text.split('/')
try:
    # args = [int(item.strip()) for item in args]
    args = [int(item) for item in args]
    ....
except Exception as e:
    print(e) # 输入有误
```

输入输出

- **eval(source)**: 分析字符串source中的表达式，返回一个该表达式对应的对象
 - eval('123')返回整数123
 - eval('1+2')返回3
 - eval('1,2')返回元组(1,2)
- **print(value1, value2, ..., sep=' ', end='\n', file=sys.stdout)**:
 - 注意sep和end的含义
 - 输出每个参数，参数之间以sep隔开，最后输出end
 - 返回None

- 掌握eval(source)函数，分析并计算字符串source中的表达式，返回该表达式运算后的值
- 掌握print()函数的用法，掌握sep/end的含义，返回None

模块：import 语句

- 模块导入：
 - **import module [as alias]**
 - 如果该module首次导入，寻找模块源文件，加载模块，创建模块对象
 - module或者alias与该模块对应的模块对象绑定
 - **from module import obj [as alias]:** 从模块中导入特定的对象，即obj与对象绑定
 - **from module import ***
- 模块的__name__属性：
 - 作为script运行时，__name__等于'__main__'
 - 否则被import时，__name__为模块的名字

- 掌握import语句：从哪个模块中导入，把什么导入到当前名字空间？
- 掌握名字 __name__

主要内容

- 基础知识：数据类型、赋值语句、运算符、内置函数、模块
- **程序结构：函数、分支结构、循环结构、异常处理**
- 数据结构：
 - 字符串
 - 列表
 - 元组
 - 字典
 - 集合
- 文件
- 正则表达式

函数

func(position-only, /, position-or-keyword, *args, keyword-only, **kwargs)
 仅位置参数传递 位置或关键字参数传递 仅关键字参数传递

- def func(...): 生成函数对象，赋值给func

• 实参与形参的定义和匹配

- 形参：位置形参、默认值形参、可变长度位置形参、仅关键字传递形参（*之后的参数）、可变长度关键字形参
 - 仅位置传递的形参：/之前的参数
- 默认值形参：函数调用时，不传递值，则表示使用默认值
- 实参：位置实参、关键字实参
- 形参和实参的顺序：都是位置形式的参数在前
- 首先匹配位置实参，然后是关键字实参，接下来剩下的位置实参收集到可变长度位置形参（tuple），剩下的关键字实参收集到可变长度关键字形参(dict)，如果还有形参尚未传递，是否有缺省值
- 函数调用采用pass by assignment的传递机制：形参=实参
 - 形参指向实参所指的同一个对象
 - 默认值形参：指向保存在函数对象的__defaults__属性中的同一对象
 - 传递的实参为可变对象：函数体内可修改该可变对象
 - 默认值形参的默认值为可变对象：上一轮调用使用默认值时，相当于传递了该可变对象
 - 默认值改造为不可变对象 def f(..., arg=None): if arg is None: arg = []
- return语句，没有时相当于 return None
- 函数体的第一行为字符串表达式时会被保存在函数对象的__doc__属性中

函数调用的序列解包与函数定义时的可变长参数

- 函数定义：可变长位置和关键字参数收集多余的位置参数或关键字参数组成元组或字典对象

```
def func(*args, **kwargs)
```

- 函数调用：序列和映射解包将可迭代对象或可映射对象展开为位置参数或关键字参数

func(*seq, **dict): 调用时实参进行序列解包：

- *seq: seq的元素展开为位置实参
- **dict: dict的元素变为关键字实参，要求字典的key为字符串，且内容为合法的变量名，展开为key1 = value1, key2=value2

- 掌握函数定义和函数调用的方法
- 掌握实参与形参的定义和匹配方法
 - 掌握默认值参数
 - 掌握可变长位置参数和关键字参数
- 掌握函数调用的序列解包
- 掌握函数作用域的LGB规则
- 掌握global语句
- 不要求nonlocal语句和嵌套函数时的作用域搜索

名字空间

- **命名空间 (namespace)** 是名字 (变量) 和对象的映射, python采用dict来实现
- 全局命名空间指的是在模块内部定义但是不在函数内部定义的名字空间
- 局部命名空间指的是在函数内部定义的名字空间
 - 允许嵌套函数, 但不作要求
- 一个变量 (名字) 如何加入到某个名字空间?
 - 只有赋值类语句、global和nonlocal (不作要求) 才会建立或者改变变量的作用域(namespace)
 - 如果在函数内有赋值, 则该名字属于本地名字空间
 - 除非通过global var声明其为全局空间的名字
 - 除非通过nonlocal var声明其为某个外层函数名字空间的名字
 - 如果在函数外赋值, 则该名字属于全局名字空间

LEGB规则

名字的命名空间（作用域）搜索：LEGB规则

- 掌握LEGB规则就可以，即首先搜索函数内部的本地命名空间，如果没有则搜索全局的命名空间，最后搜索内置名字空间
- 在访问某个名字时，根据语句所在的位置决定查找的起始点，但是如果在函数体内：
 - 如果显式说明为global，则仅仅在顶层名字空间查找
 - （不要求）否则如果显式说明为nonlocal，则只在外层函数名字空间中查找
 - 否则如果在函数体内有赋值语句(不管在前面还是后面)，则只在local查找
 - 否则按照LEGB顺序查找

闭包函数（不要求）

函数体内定义一个内层函数时，如果满足下述情形，则称为闭包函数

- 内层函数访问外层函数中的变量，假设为x，
 - 内层函数体内没有对x赋值，按照EGB顺序查找，定位到某个外层函数
 - 内层函数体内需要对x赋值，引入nonlocal x显式声明，定位到某个外层函数
- 引入cellvar和freevar进行间接访问：
 - 外层函数引入cell变量x，要访问x或者对x赋值时，采用间接访问：x ---> cell ---> object
 - 内层函数引入自由变量x，其属性__closures__保存了cell，在调用该函数时，设置自由变量x，采用间接访问：x --> cell --> object
- 对于程序员来说，并不需要详细了解内部实现的细节，只要知道：内层函数可以访问外层函数中的变量x，即便在该外层函数的调用已经结束也是如此，而且x也只能通过内层函数才可以访问

分支(选择)结构: 条件表达式

- 任何表达式都是条件表达式, 可用于选择和循环结构中
 - 表达式不能包含赋值类语句
- 条件表达式的真值判断: 对象一般真值判断为真:
 - False: 值为0、None、空序列对象 (长度为0)
 - True: 非0、非None、非空对象
- 逻辑运算符: and or not, 注意**短路逻辑**
 - and: 返回第一个假(None、空或者数值0)的表达式或者最后一个表达式 name or default
 - or: 返回第一个真 (非None、非空或者非0) 的表达式或者最后一个表达式 name and alternate
 - not: not True = False, not False = True
 - 主要**考察逻辑运算用于选择或循环结构中作为条件判断的情况**
- **逻辑运算符: 二选一的if else三元表达式:** `value = x ** 2 if x % 2 == 0 else x ** 3`
- **德摩根定理(De Morgan's Law)**
 - `not (expr1 and expr2) = not expr1 or not expr2`
 - `not (expr1 or expr2) = not expr1 and not expr2`

- 掌握对象的真值判断方法
- 掌握 and/or/not/if else逻辑运算符, 什么情况下真值判断为真?
- 掌握短路逻辑的概念
- 掌握德摩根定理和分配律
- 了解and/or运算的最终结果

分配律: `expr1 and (expr2 or expr3) = (expr1 and expr2) or (expr1 and expr3)`

`expr1 or (expr2 and expr3) = (expr1 or expr2) and (expr1 or expr3)`

选择结构

- 单分支、双分支和多分支
- 选择结构的嵌套，注意缩进

- 掌握选择结构，包括单分支、双分支以及多分支

```
if expr:
    statement1
elif expr2:    # 前面条件不满足时
    statement2
elif expr3:
    statement3
else:
    statement_else
```

range函数

range(stop):

range(start,stop[,step])

- 返回一个range对象，可产生一系列的整数，从start（缺省为0）开始，直到stop为止（不包括），step为步长，缺省为1

range(10)

range(1, 100, 2)

range(10, 0, -1)

- 掌握range对象的使用方法
- range(n) 产生n个数值，可用来重复n次
- range(len(s)): 产生s中每个元素对应的下标
- range(len(s)-1, -1, -1) 或range(-1, -len(s)-1, -1) 逆序的下标

循环结构

- while循环和for循环
- break和continue语句的含义
- else在什么情况下执行？循环不是通过break跳出时才执行
- 理解循环嵌套

- 掌握break和continue语句
- 掌握else子句
- 掌握循环变量的变化，什么时候结束循环

```
while 条件表达式:
    循环体
    if 条件表达式1: break #可选
    # Exit loop now, skip else if present
    if 条件表达式2: continue # 可选
    # Go to top of loop now
else: #可选
    # Run if we didn't hit a 'break'
    else子句
```

```
for target in iterable: # Assign iterable items to target
    循环体
    if 条件表达式1: break #可选
    # Exit loop now, skip else if present
    if 条件表达式2: continue #可选
    #go to top of loop now
else: #可选
    else子句 # If we didn't hit a 'break'
```

循环结构

问题求解：

- 首先确定步骤
- 将每个步骤转变为python代码
- 对于多分支情况：确定条件表达式以及相应分支
- 对于要执行多次情况：采用循环结构
 - 循环次数是否已知： for 或者 while expr
 - 确定循环结束的条件：
 - if expr: break
 - while not expr:
 - 确定循环继续条件： continue
 - 不要忘记更新循环变量

问题：取整数的各位上的数字

- 实现1
 - loop: 从最低位开始取，重复以下步骤，直到当前数为0：
 - 使用%运算，取最低位数字
 - 使用//运算，取剩下的数字
- 实现2：转换为字符串，然后取每个字符，再转换为整数，可用列表推导式实现

- 边界条件是否正确？是否有偏—错误？是否遗漏了特殊情形
- while循环体的各个执行路径是否更新了循环变量？
- for item in iterable 的循环体中，不要改变原有对象的大小（增加或者删除元素）
 - iterable为列表时：选择合适的遍历顺序；采用while循环
 - iterable为字典时：分阶段，首先遍历确定要删除的key，然后执行删除动作

循环结构

- 栅栏循环：用户连续输入整数，每次输入后进行计算，直到用户输入空字符串结束
- 用循环解决级数求和问题
 - 项数之间的关系: 对于第n项，等于前一项/n --> $item /= n$
 - 循环变量n，初始值为1，和的初始值为1，结束值未知，结束条件为 $item \leq error_factor$
 - 循环变量n每次加1

- 掌握使用循环结构解决实际问题的方法
- 掌握用户输入合法整数的方法
- 掌握遍历容器对象，对于每个元素（还要知道是第几个元素）进行处理的方法
- 掌握级数求和问题的解决方法
- 掌握每行最多输出5个值的实现方法

```
while True:
    text = input('....')
    if not text:
        break
    try:
        arg = int(text.strip())
        print(arg)
    except Exception as e:
        print(e)
```

```
approx = item = 1
n = 1
while True:
    item /= n
    approx += item
    if item <= error_factor:
        break
    n += 1
```

异常处理

- 异常：运行时错误，导致程序运行出错而跳出正常控制流
- 异常处理：捕获异常，进行相应的处理

- 常用的异常类：Exception

- StopIteration
- SyntaxError
- ZeroDivisionError: 除数为0
- NameError, ValueError
- IndexError, KeyError
- AssertionError
- EOFError: 输入结束时触发

- raise语句：

- raise ValueError("...")
- raise ValueError
- 出现在except block中：
 - raise
 - raise ValueError("...")
 - raise ValueError("...") from None

- 了解什么是异常
- 掌握异常处理中异常匹配的方法，Exception是一般情况下用户捕获的异常的大类
- 掌握使用raise子句抛出指定类型和原因的异常
- 了解异常处理中没有异常，出现异常但是捕获、出现异常但没有捕获时的代码段执行轨迹
- 了解finally部分出现return/break/continue等跳转类语句会取消异常的情况
- 掌握使用异常处理提高程序鲁棒性的方法，比如要求用户输入整数或浮点数
- 了解assert语句和traceback模块

异常处理

- finally: 不管有否异常都要执行
- else: 前面没有异常出现时执行
- except: 按照顺序匹配, 如果匹配, 则执行, 后面的except跳过
 - except: 等价于except BaseException:
- 如果出现异常但是未捕获, 则try语句后面的代码不会执行, 向外层抛出异常

```
assert expression[,reason]
```

```
if __debug__:  
    if not expression:  
        raise AssertionError  
        # raise AssertionError(reason)
```

```
try:  
    <body>  
finally:  
    <finallyBody>
```

```
try:  
    <body>  
except [expression [as identifier]]: # 至少一次, 在多次出现时不带表达式except应该是最后一个。  
    按序匹配找到对应的exception为止  
    <exceptionBody>  
else: # 可选的, 在没有异常出现时执行  
    <elseBody>  
finally: # 可选的, 不管异常有没有都要执行  
    <finallyBody>
```

主要内容

- 基础知识：数据类型、赋值语句、运算符、内置函数、模块
- 程序结构：函数、分支结构、循环结构、异常处理
- **数据结构：**
 - 字符串
 - 列表
 - 元组
 - 字典
 - 集合
- 文件
- 正则表达式

字符串

- 字符串字面量
 - 单引号、双引号、三单双引号 (可以跨越多行)
 - 字符转义: `\n \t \\ \'`
 - 原始字符串: 字面量定义前添加`r`或`R`
 - 比如`r'c:\user\xxx\sample.txt'`

- 字符串运算: `+` `*`

`'abc' + str(4);` `'abc' * 3`

- 常用的字符串相关内置函数:

- `chr`、`ord`

- `int/float/str`

a	b	c	...	x	y	z
A	B	C		X	Y	Z
0	1	2	...	23	24	25

- 掌握字符串字面量定义的方法, 包括原始字符串
- 掌握基本的字符转义方法
- 掌握字符串运算 `+` `*`, 以及常用内置函数(`chr/ord/int/float/str`)
- 掌握字符串的常用方法:
 - 去除首尾空格(字符): `strip`
 - 判断字符串类型: `isspace/isdigit/isalpha`等
 - 判断字符串的前后模式: `startswith/endswith`
 - 字符串的大小写转换: `lower/upper`
 - 查找: `count/index/rindex/find/rfind`
 - 分割和合并: `split`、`splitlines`和`join`
 - 替代: `replace/maketrans + translate`
- 掌握字符串的格式化方法: `format`方法和`f-string`
- 掌握数值对象与字符串之间的转换(`int,float,str,bin,oct, hex`)

小写变为大写: `chr(ord(ch) - ord('A') + ord('a'))`

大写变为小写的另一种方法: `alphabet='abcdefghijklmnopqrstuvwxyz'`
 `lower = alphabet[ord(ch) - ord('A')]`

字符串：方法

- 通用序列方法：下标、切片、in、len、sorted、reversed、enumerate、zip
- 去首尾(空白) 字符：strip([chars]) lstrip([chars])rstrip([chars])
 - 去除字符串前后的某些字符。
 - 缺省是去除空白类字符(空格、制表、换行等)

```
text = input('...')
text = text.strip()
args = text.split(',')
args = [item.strip() for item in args]
```

- 分割单词时，如果要去除前后的标点符号

```
text = input('.....')
text = text.strip()
words = text.split()
words = [word.strip('\n"?.!') for word in words]
```

字符串：方法

- 字符串类型判断: isdigit isalpha isspace islower isupper

- 判断字符串的**每个字符**是否是空格,数字,字母等
- 判断字符串的**每个字符**是否有大小写概念, 如果有是否为小写,大写

```
if not line or line.isspace():  
    print('empty line')
```

- 大小写转换: lower upper

- 测试和查找:

- startswith(prefix) endswith(prefix)
- count(sub) index(sub) rindex(sub)
- find(sub) rfind(sub): 注意find和index的区别, 是否抛出异常

```
text = input('continue..? y or n')  
if lower(text) == 'y':  
    print('bye')
```

```
pos = strobj.find(substring)  
if pos != -1:  
    print(strobj[:pos], substring, strobj[pos + len(substring):])
```

```
try:  
    pos = strobj.index(substring)  
    print(strobj[:pos], substring, strobj[pos + len(substring):])  
except Exception as e:  
    print('not found')
```

字符串：方法

- **拆分和组合**: `split()` `splitlines()` `rsplit(sep)` `split(sep)` `join(iterable)`
 - 了解 `partition(sep)` `rpartition(sep)`
 - 暂时认为: `split('\n')`和`splitlines()`都可以使用
- **替代**: `replace(old, new, count=-1)`
- **填充和对齐**: `center` `ljust` `rjust(width[,fillchar])`
- **翻译和转换**: `maketrans/translate`

`split(sep)`以`sep`分割字符串，分割后可能出现空字符串，分割后的字符串可能前后有空格

```
text = input('...')
args = text.split(',')
args = [item.strip() for item in args]
args = [item for item in args if item]
```

可使用`join`方法将字符串列表(可迭代对象)合并成字符串

```
str_list = [str(i) for i in range(10)]
text = '+'.join(str_list)
```

```
text = input('....')
text = text.replace(',', ' ')
str_args = text.split()
```

```
tab2 = str.maketrans('123456789', '*'*9)
s1.translate(tab2)
```

格式化

```
"First {0:s}, Second {1:d}, Third {2:5.2f}".format('idle',90,93.4)
f"First {'idle':s}, Second {90:d}, Third {93.4:5.2f}"
```

`format_string.format(value1, value2, ...)`

- {field:[fill][align][width][.precision]type}
- 对齐: <左对齐, ^居中对齐, >右对齐
- ,或者_ 数值类型格式化时每千位添加逗号或者下划线

`f-string: f'...{expr:format_spec}...'`

动态构造

```
'{0:{1}}'.format(12345, 10) # '{0:10}'.format(12345)
'      12345'
```

- 掌握格式化
 - 如何指定最小宽度、对齐方式和填充字符?
 - 如何指定精度?
- 了解通过格式化嵌套动态决定如何格式化
- 了解字符串的填充对齐方法:
 - center/ljust/rjust/zfill

format格式	说明
{:s}	输出字符串
{:10s}	格式化为字符串, 最小宽度为10
{:5d}	将整数格式化为总宽度至少为5的十进制形式, 不够填充空格
{:10.2f}	格式化浮点数, 总宽度为至少10, 四舍五入到小数点后第2位
{:<d}	缺省右对齐, -表示左对齐

数值对象与字符串的转换

- `int(...)`:
 - `int(number)`: 数值类型对象取整：往原点方向取整，不是四舍五入！
 - `int(x[,base])`: 将指定进制的整数字符串转换为整数
- `float(...)`:
 - `float(number)`: 数值类型对象转换为浮点数
 - `float(x)`: 将十进制整数、十进制小数形式、十进制科学计数法形式的字符串转换为浮点数
- 格式化:
 - `'{:d}'.format(value)`; `format(value, 'd')`; `f'{value:d}'`
 - `'{:.4f}'.format(value)`; `format(value, '.4f')`; `f'{value:.4f}'`
 - `.4f`, 指定精度，四舍五入后变为字符串
- `round(number, n)`: 四舍五入，仍然为数值类型

了解 random模块

seed(a=None)	给出随机数生成器的seed
randint(a,b)	返回在[a,b]区间的随机整数
randrange(start, stop=None, step=1)	从range(start, stop[, step])中随机选择一个整数
random()	返回在[0,1)区间的随机实数
uniform(a,b)	返回在[a,b]区间的随机实数
choice(seq)	返回非空有序序列seq中的（随机选择的）某个元素
shuffle(listobj)	列表listobj原地随机排序，返回None
choices(seq, weights=None, *, k=1)	从非空有序序列seq中随机(返回)选择k个元素组成一个新列表。weights可传递seq中每个元素的权重，否则每个元素被选中的机率相同
sample(seq, k)	从非空有序序列seq随机(不放回)选择 k个元素组成一个新列表

python数据结构概述

- 无序和有序：
 - set、dict为无序对象。 **set可去除重复元素**
 - list、tuple、str、range为有序序列：
 - 可以通过下标、切片等来访问序列中的元素
- str的元素为单个字符，而list、tuple、set、dict的元素可是其它python对象
 - set要求其元素为不可变对象
 - dict要求其key为不可变对象
- 可变和不可变：
 - list、set、dict为可变序列
 - tuple、str为不可变序列，其中**tuple对象不可改变其顶层数据，即元素不可添加、删除、修改，但是其元素所指向的对象本身的值可以修改**

- 掌握序列对象的区别，是否有序、是否可变等
- 有序对象(包括range对象)可以通过下标和切片访问

创建序列或映射对象

list、tuple、dict、set的定义方式

- 调用构造函数: `list([iterable])`、`tuple`、`dict`、`set`
 - 其中dict传递的参数为iterable时, 要求每个元素必须包括两个子元素
 - `dict(**kwargs)`也可以采用关键字参数, 参数名作为key, 值作为value
 - `dict.fromkeys(seq[,value])`: 序列中的元素作为key, 值为value, 缺省为None
- 字面量(Literal)定义:
 - 分别为[]、()、{}、{ }
 - 元素中间用,分割
 - 考虑到()也可用于表达式, 如果tuple只有一个元素时, 添加逗号, 即(1,)
 - dict和set都采用{}, 因此空set不能用literal定义, 而是set()
 - dict的每个元素用 key:value的形式表示, key必须是immutable对象, 但是immutable对象不一定可以作为key, value可是任意python对象

- 掌握各个序列对象的字面量定义
- 掌握各个序列对象的构造函数方法, 注意字典对象作为iterable对象时, 等价于调用其keys()方法

有序对象：下标和切片

0	1	2	3	4	[0,5)
10	20	30	40	50	len(s)=5
-5	-4	-3	-2	-1	[-5,0)

- 有序对象的下标：
 - 下标从0开始，到len(s)-1为止
 - 负数下标从-1开始到 -len(s)为止，与正数下标的关系为加上len(s)
- 切片访问：[start:stop] or [start:stop:step]
 - step省略，表示缺省为1
 - 如果下标i小于0，则可规范为 i + len(s)
 - 如果**step大于0**，从下标为start的元素开始按照顺序**往后**直到下标为stop的元素为止（不包括下标为stop的元素）
 - start可省略，表示下标为0
 - stop可省略，表示下标为列表结束后的下一个位置，即len(s)
 - 如果**step小于0**，则从下标为start的元素开始，按照相反的顺序**往前**直到下标为stop的元素为止（不包括下标为stop的元素）
 - start可省略，表示下标为-1或者len(s)-1
 - stop可省略，表示下标为-len(s)-1
 - start/stop可以不在合法下标范围，**会自动截取到合适的位置（start和stop的缺省位置）**
 - 切片出现在RHS时表示返回的是原有序对象的相应元素组成的新的有序对象，比如 new_lst = lst[:]

s[:] 有序对象的copy
s[::-1]: 逆序
s[1:-1]: 去掉第一个和最后一个
s[0::2]: 偶数位置
s[1::2]: 奇数位置

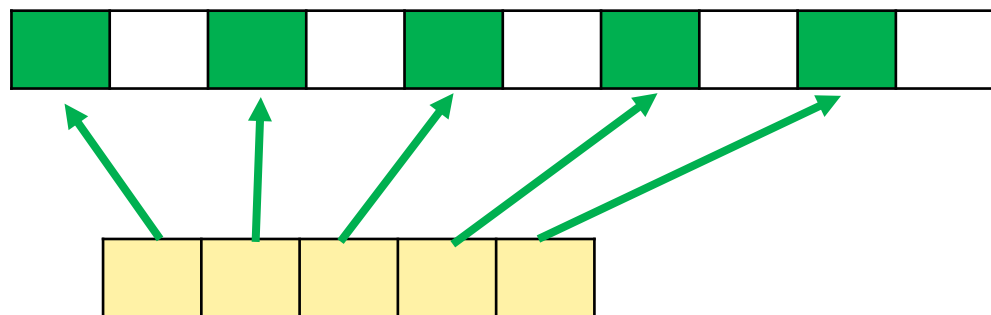
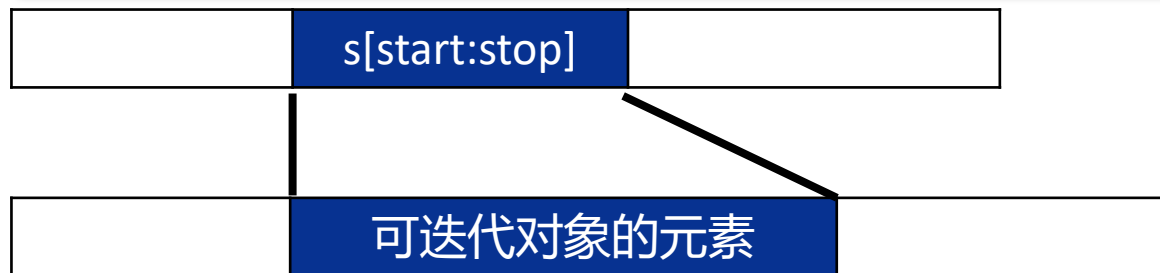
idx < 0时，相当于0
idx > len(s)时，相当于len(s)

idx > len(s)-1时，相当于len(s)-1
idx < 0时，相当于-len(s)-1

切片：原地修改列表

- 切片出现在表达式处，表示根据指定位置的元素构造一个新的同种类型的有序对象
- 切片出现在赋值语句左边，表示原地修改(插入、替换或删除)列表中的元素

在下标 i 处插入多个元素: `list_var[i:i] = iterable`



步长不为1时要求左右两边的元素个数一样
→ 无法通过赋值来删除不连续的多个元素

切片可以适用于有序对象

- 掌握切片的两种格式，两种格式中哪些可以省略，表示什么含义: `s[1:3]`, `s[1:5:2]`
- 掌握使用切片原地修改列表的方法
- 掌握`del`语句删除列表中的元素或者多个元素的方法

替换或者删除元素

`s[start:stop] = iterable`

`s[::2] = iterable`

删除列表中的多个元素

`del s[start:stop]`

`del s[start:stop:step]`

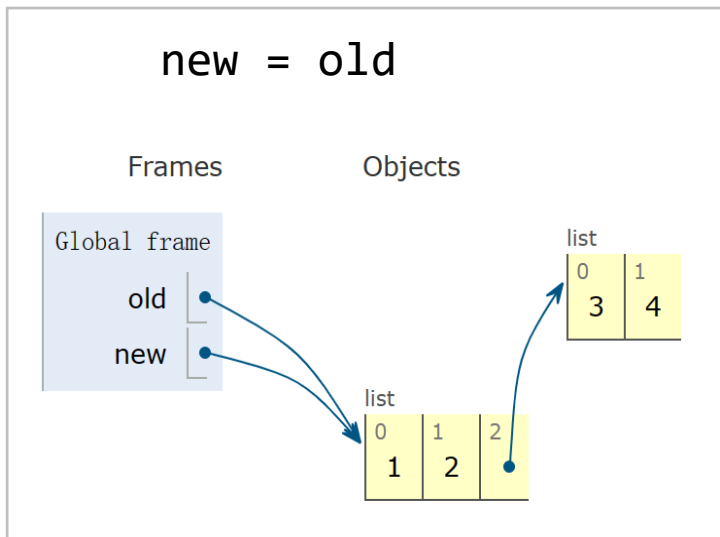
有序对象的通用方法

- 有序序列支持：下标和切片
- 成员关系判断：in not in (注意与is is not的区别)
- index和count，返回元素在序列中的下标和出现次数，不存在时会抛出ValueError
- 运算符：
 - + 创建新的序列，两个序列元素合并
 - += 对于列表，表示原地合并列表
 - * 其中一个参数为整数，表示重复序列的元素
 - *= 对于列表，表示原地重复
- 比较运算：
 - 同种类型的对象的比较，但注意数值类型可以进行类型提升： $1 < 1.45$
 - 不是所有的对象都支持比较，比如complex、dict不支持
 - 序列的比较为对应元素的比较，直到确定关系为止： $(2,3) < (2,4,5)$ $(2,4) < (2,4,5)$
 - set的比较运算为子集，真子集的运算

- 掌握有序对象的通用方法
- 掌握哪些对象可以比较：complex与dict无法比较
- 掌握有序对象的比较运算

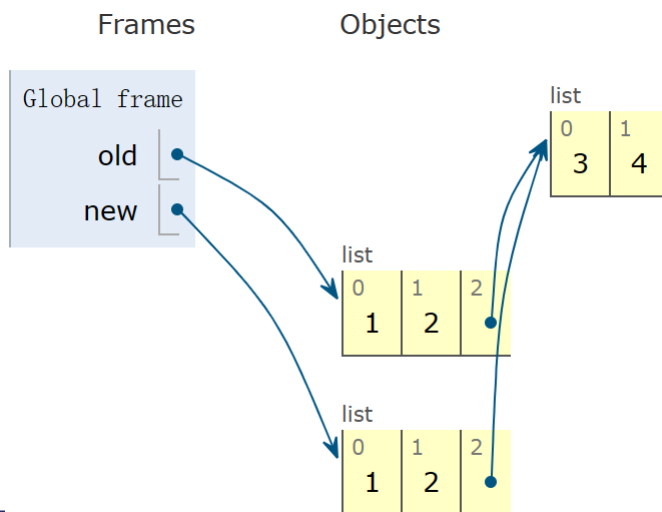
浅拷贝

- `new = old`: 给原有对象取了一个别名
- 浅拷贝: 新的对象与原始对象共享相同的子对象
- 深拷贝: 新的对象与原始对象完全独立



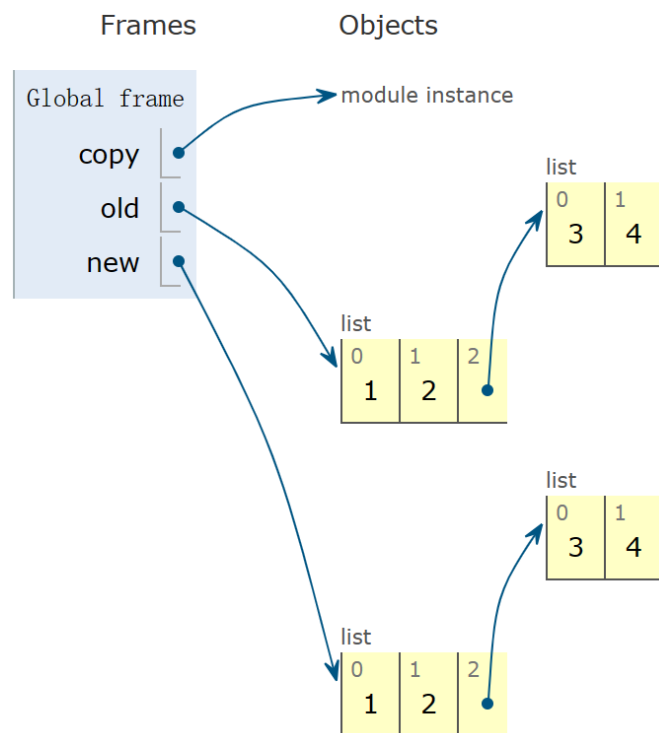
`import copy`

```
old = [1, 2, [3, 4]]  
new = old[:]  
new = old.copy()  
new = copy.copy(old)
```



`import copy`

```
old = [1, 2, [3, 4]]  
new = copy.deepcopy(old)
```



可迭代对象（包括有序对象）的常用内置函数

- `reversed(iterable)`:
 - 返回一个迭代器对象，为可迭代对象的各个元素逆序
 - `obj[::-1]` 有序对象可通过切片获得其逆序
- `sorted(iterable, reverse=False, key=None)`: 排序后返回新的list
 - 如何定义函数或采用lambda表达式进行排序
- `len(iterable)`、`max(iterable)`、`min(iterable)`: 长度，最大值，最小值
 - `max`和`min`也可以如同`sorted`一样，传递`key`参数，作为求最大最小值的基准，如 `max(text.splitlines(), key=len)`
- `sum(iterable)`: 数值元素的和
- `enumerate(iterable[,start])`: 返回枚举对象，每次返回包含下标和值的元组
 - `start`缺省为0
 - 用于遍历`iterable`对象，在遍历的同时也知道是第几个元素
 - 除了采用`enumerate`外，还可以采用 `for i in range(len(s))`，或者引入变量`idx` + `for/while`来实现上述功能
- `zip(iter1,iter2...)`: 返回zip对象，每次返回各个可迭代对象中对应的相同位置的各个元素组成的元组，相当于`zip shortest`.
 - 不要求：如果希望`zip longest`呢？可使用`functools`的`zip_longest`方法，或者自己填充短的，使得一样长：
`list(iterable1) + [None] * (len(iterable2) - len(iterable1))`

• 掌握这些常用内置函数的用法，特别是`sorted/enumerate/zip`函数

• 了解`zip_longest`方法

列表的常用操作

除了index, count, pop等，其他方法返回值都是None

- list.append(x)和list.extend(L)：注意两者的区别
- list.reverse()和list.sort()：原地逆序和排序操作，与内置函数reversed和sorted的区别

- 掌握增加和删除列表元素的方法
- 了解通过复合赋值(+= *=) 原地修改列表的方法

列表元素增加方法	别名	效果
list1 + list2	拼接	新建列表
append(obj)	附加	原地修改
extend(iterable)	扩展	原地修改
insert(index, obj)	插入	原地修改
list1 * n	复制	新建列表
了解 list1 += iterable	复合赋值	原地修改
了解 list1 *= n	复合赋值	原地重复

方法	作用	返回	备注
del list_obj[index]	删除指定位置元素	无返回值	IndexError,TypeError
pop([index])	删除指定位置元素并返回该元素	返回元素	IndexError,TypeError
remove(value)	删除首次出现的指定元素, 返回None	返回None	没有相应元素时ValueError
clear()	删除所有元素	返回None	空列表

列表解析式和生成器表达式

[表达式 for 变量 in 列表] 或者 [表达式 for 变量 in 列表 if 条件]

- 对列表中的每个元素，在满足条件的情况下，计算表达式的值作为新列表的元素
- 表达式可以包括函数调用等

```
[x**2 for x in range(100) if x%3 == 0]
```

```
[weapon.strip() for weapon in freshfruit]
```

```
[(x,y,z) for z in range(100) for y in range(1,z) for x in range(1,y) if x*x + y*y == z*z]
```

```
{k:len(v) for k, v in d.items() if k > 0}
```

- 掌握列表解析式(包括生成器表达式、字典解析式等) 的写法，要求掌握1到2个for子句的情形
- 了解生成器表达式与列表解析式的区别

- **生成器表达式**的写法与列表推导式基本类似，只是用圆括号，但是：
 - 列表推导式计算的结果是**新的列表**，一次生成所有的元素
 - 生成器表达式的结果是一个生成器对象，**符合迭代器协议（打开的魔盒）**，调用next时才会实时返回下一个元素

```
gen = (i*2 for i in range(10))
```

```
for item in gen: print(item)
```

```
for item in gen: print(item)    # 不会有任何输出
```

序列解包

LHS = RHS

- LHS为对象引用，变量或者下标切片描述的多个list元素
- LHS允许出现*seq，但只允许出现一次
 - 该引用前后的变量——对应赋值后，剩余的变量转变为list然后赋予该引用
- RHS可以是任何可迭代对象，包括tuple、list、dict、range、str等，逐个取该序列的元素赋予左边对应位置的对象引用
- 除*seq以外的LHS的变量引用与RHS的元素必须——对应
- 嵌套序列解包： `index, (key, value) = 1, ('name', 'tony')`
- 交换a和b： `a, b = b, a`
- 与eval结合在一起进行输入： `a,b,c = eval(input('Please input a,b,c'))`

- 掌握赋值语句中的基本序列解包以及嵌套序列解包
- 掌握函数调用时的序列解包
- 掌握序列解包可用于交换元素
- 掌握扩展序列解包

```
a, b, c = [1, 2, 3]
a, *b, c = range(1, 7)
for index, value in enumerate(dList):
    print('%d:%s' % (index, value))
```

```
for index, (key, value) in enumerate(dictobj.items(), 1):
    print('%d. %s:%s' % (index, key, value))
```

序列解包的各种情形

使用场景	描述
<code>x, y, z = range(3)</code>	赋值语句的基本序列解包，支持嵌套序列解包
<code>x, y, z, *seq = range(10)</code>	赋值语句的扩展序列解包 ，支持嵌套序列解包。首先匹配其他位置的对象引用，剩余的元素作为seq所指向的新列表中的元素
<code>func(x, y, *seq)</code>	函数调用时的序列解包，将可迭代对象seq的各个元素拆分后作为位置实参
<code>func(x, y, *seq, **map)</code>	函数调用时的序列解包，表示将map对象(如字典)的各个元素拆分后作为关键实参 (var=value)
<code>[*range(4), 4]</code>	元组、列表、集合字面量定义时的序列解包，将可迭代对象的各个元素拆分后作为新的对象中的元素
<code>{'x': 1, **{'y': 2}}</code>	字典字面量定义时的序列解包，将map对象的各个元素(key:value)拆分后作为新的字典对象中的元素

可变长参数

使用场景	描述
<code>def func(*args, **kwargs): print(args) print(kwargs)</code>	函数定义时可变长度位置参数以及可变长度关键字参数，收集调用时传递的尚未匹配的位置参数以及关键字参数，以元组和字典形式保存

字典

- key in d
- 字典元素的读取：
 - d[key] 以键作为下标可以访问字典元素，若键不存在则抛出异常`KeyError`
 - d.get(key[,default]): key不存在时返回0
- 字典元素的添加与修改：`d[key]=value`
 - d.update(another)
 - d.setdefault(key, default): update + get结合
- 字典元素的删除：
 - del d[key]
 - d.pop(key)/d.pop(key, value)
 - reverse = d.pop('reverse', True)
 - d.popitem()
 - d.clear()

- 掌握字典如何初始化，如何添加或修改元素
- 掌握字典的value为不可变对象时基本的更新方法，特别是get()方法
- 掌握字典的value为可变对象时基本的更新方法

字典

如果字典的value保存的为不可变对象

```
d = dict() # d = {}

d[key] = d.get(key,0) + 1

if key in d:
    d[key] += 1
else:
    d[key] = 1

try:
    d[key] += 1
except KeyError:
    d[key] = 1
```

如果字典的value保存的为可变对象，比如为列表

```
current_or_init_value = d.get(key, [ ])
current_or_init_value.append(value)
d[key] = current_or_init_value
```

```
d[key] = d.get(key, [ ])
d[key].append(value)
```

```
if key not in d:
    d[key] = [value]
else:
    d[key].append(value)
```

```
try:
    d[key].append(value)
except KeyError:
    d[key] = [value]
```

```
d.setdefault(key, [ ]).append(value)
```

最不推荐的方法

```
d[key] = d.get(key, [ ]) + [value]
```

字典的遍历

- `keys()`: 返回可迭代对象, 元素为字典中的所有key
- `values()`: 元素为所有value
- `items()`: 元素为(key,value)元组
- `iter(d)` or `for key in d`: 等价于`iter(d.keys())`: 元素为所有key

- 掌握遍历字典中的元素的方法, 知道key可以得到其对应的value
- 掌握字典与sorted、切片结合在一起, 实现类似于topn的功能

字典的知识点:

- 怎么样更新或者添加字典元素? `dictobj[key] = ....`
- 更新字典元素:
 - `dictobj[key] = dictobj.get(key, value) + xxxx`
 - `d.setdefault(key, [ ]).append(value)`
- 要遍历字典的元素: `for key in d / for key in d.keys() d.values() d.items()`

set

- 集合的特性： **不重复且无序**，元素必须为不可变对象

• == != < <= > >= in not in len

t = [1, 2, 3, 1, 4, 1]

- add(x), remove(x), discard(x)

怎么去除重复对象？

s = list(set(t))

s = list(dict.fromkeys(t))

怎么判断字符串中字符是否有重复？

len(set(t)) == len(t)

将集合a中的那些属于集合b中的元素移走： {'reverse', 'key', 'option'} - {'reverse', 'key', 'verbose'}

- 掌握集合的特性
- 掌握如何利用集合的不重复特性去除重复对象和判断是否有重复

sorted

L.sort(key=None, reverse=False) 原地排序列表, 返回None

sorted(iterable, key=None, reverse=False) 对可迭代对象的元素排序, 返回新的列表

max(iterable, key=None) min(iterable, key=None)

- 新列表的元素来自于可迭代对象
 - 传递的参数为元组、列表、集合等, 则新列表中的元素就是这些对象中原来的元素
 - 传递的参数为字典, 新列表中的元素就是字典对象的key
 - 通过key可以得到其对应的value。sorted传递的key参数可以利用这一点
 - 传递的参数为dictobj.items(), 排序后的元素为元组(key, value)
- 排序的基准是调用关键字参数所传递的函数key(item)得到的结果
 - 经常采用lambda表达式传递函数对象
 - 可以利用元素本身的大小关系
 - 有多个排序基准时, 计算排序基准的函数对象返回一个包含(主排序基准, 第2排序基准, ...)的元组对象
 - 采用lambda表达式时, 元组定义的圆括号不可省略
 - 在多个排序基准中, 如果升序和降序不统一
 - 某个排序基准是数值类型, 则可以利用负数运算实现排序基准的统一
 - functools.cmp_to_key(cmp_function)

- 掌握利用元素本身大小关系排序
- 掌握如何通过key进行排序
- 掌握有多个排序基准时如何排序
- 了解排序基准中, 升序和降序不统一时如何处理

```
def __anonymous__(arguments):  
    return expr
```

```
lambda arguments: expr
```

主要内容

- 基础知识：数据类型、赋值语句、运算符、内置函数、模块
- 程序结构：函数、分支结构、循环结构、异常处理
- 数据结构：
 - 字符串
 - 列表
 - 元组
 - 字典
 - 集合
- **文件**
- 正则表达式

文件操作：打开和关闭

- `f = open(file, mode='r', encoding=None)`
- `file`: 相对路径或绝对路径
- **`mode`**: 缺省为打开文本文件读
 - `rwx`: 取其中一种模式
 - `tb+`: 可选的
 - `t`或者`b`: 文本或二进制模式
 - `+`: 读写模式, 必须要指定`rwx`中任意一种
- **`encoding='utf-8'`**, 或者不指定时采用默认的编码方式
- `f.close()`: 关闭文件释放资源
- 文件操作的三种模式:
 - `open + process + close`
 - `open + try: process + finally: + close`
 - **`with open as handler: + process`**, 系统自动`close`释放资源

- 掌握内置函数`open`, 打开模式`mode`, 编码方式`encoding`
- 掌握使用`with`语句, 保证文件对象关闭
- 掌握使用`read/readline/readlines`以及文件对象本身作为迭代器时, 对于文件读的方法
- 掌握使用`write`写文件, 了解`writelines`方法
- 了解二进制文件和文本文件的区别

文件操作：读写

• read、readline和readlines的区别：

- read指定长度或者全部读取到str
 - splitlines(): 不包含分隔符（换行符）
- readline: 读取一行到str（换行符包括在内）
- readlines: 读取所有行(换行符包含在内)，返回字符串列表
- write、writelines
 - write: 写字符串
 - writelines: 字符串列表中的每个字符串写入，注意中间不会添加换行符

一行一行地读

```
with open('text.txt', 'r') as f:
    while True:
        line = f.readline()
        if not line: # 文件结尾
            break
    ....
```

一次读完，直接返回行列表

```
with open('text.txt', 'r') as f:
    lines = f.readlines()
```

文件对象为迭代器对象

```
with open('text.txt', 'r') as f:
    for line in f: # 文件对象为迭代器对象
        ....
```

一次全部读完，然后根据需要可以再分割为行进行处理

```
with open('text.txt', 'r') as f:
    text = f.read()
    lines = text.splitlines()
    # lines = text.split('\n') 稍有区别，但建议用splitlines()
```

正则表达式:

正则表达式的元字符集: ^ \$ * + ? | { } [] ()

- 字符类: `[xyz]` `[^xyz]` `[a-z]` `[^m-z]`
- 预定义字符类: `.` `\d` `\D` `\w` `\W` `\s` `\S`
- 边界匹配符: `^` `$` `\b` `\B`
- 重复限定符: `?` `*` `+` `{n}` `{n,}` `{,m}` `{n,m}`
- 分组符() `\1`
- 选择符|
- 元字符需要转义: `\*` 或者 `[*]`, 如果需要动态构造正则表达式, 可能要使用 `re.escape(pattern)`
- 正则表达式模式: 采用原始字符串 `r'....'`

- 传递正则表达式建议采用原始字符串
 - `str`字面量定义与`re`所采用的转义字符都是\
- 懒惰和贪婪匹配
 - `r'<.+?>'` 匹配 < 尽可能少匹配1个以上字符 >
 - `r'<[^>]+>'` 匹配 < 尽可能多匹配1个以上非>字符 >

- 掌握基本的正则表达式的写法: 如何描述字符集(包括预定义字符类), 如何描述某个子模式或字符的重复次数, 如何通过选择符匹配多个子模式中的任一个子模式
- 掌握正则表达式中转义字符的使用方法, 特别是 `\.` `\b` `\*` `\(` `\)` `\[` `\]` 等
- 了解`re`模块的`escape`方法

正则表达式使用方式

- 直接调用re模块的方法
- 首先compile得到模式对象，然后调用该方法

```
regexp = re.compile(pattern, flags)
```

- 传递正则表达式参数，由于正则表达式也使用\作为转义。建议采用原始字符串，原始字符串的内容就是要写的正则表达式
- 查找模式：search方法在text中查找第一个匹配的模式，返回match对象。match方法判别text最开始是否为对应的模式，如果是，返回match对象
- **regexp.finditer(text): 返回一个迭代器，其元素为所匹配的match对象（方法：group, start, end, span）**
- **re.search(pattern, text, flags) re.match(pattern, text, flags) , 返回match对象，或者None(没有找到)**

```
regexp.search(text, pos=..., endpos=...)
```

```
regexp = re.compile(r'\b((\d{4})/(\d{1,2})/(\d{1,2}))\b')
pos = 0
while True:
    match = regexp.search(text, pos)
    if not match:
        break
    print(match.group())
    pos = match.end( )
```

常用的flags:

- re.I re.IGNORECASE 大小写无关匹配
- re.S re.DOTALL .可匹配换行符
- re.M re.MULTILINE 多行匹配，即^ \$可匹配行首和行尾
- re.U和re.A 预定义字符集所对应的字符集

```
for match in regexp.finditer(text):
    print(match.group(1))
```

正则表达式使用方式

`re.match(pattern, text, flags)`: pattern出现在text的开始位置

`re.fullmatch(pattern, text, flags)`: pattern与text正好匹配

`re.findall(pattern, text, flags)`

`regex.findall(text)`

- 返回一个列表，该列表中的每个元素描述了某次匹配的内容
 - 如果模式中有分组符，即包含子模式，则
 - 匹配的内容仅仅包含了子模式
 - 如果有多个子模式，则用元组方式组织
 - 注意不包括模式0(即整个匹配的内容)，如果想要整个匹配的内容，则引入（第1个）分组来描述整个模式，编号也相应改变
 - 如果模式中没有分组符，则包含整个模式匹配的内容

```
dates = re.findall('\b((\d{4})/(\d{1,2})/(\d{1,2}))\b', text)
dates = [date[0] for date in dates]
```

正则表达式使用方式

re.split(pattern, text, maxsplit=0, flags=0)

regex.split(text, maxsplit=0)

- 查找模式pattern作为分隔符来分割字符串

```
text = input('...')
words = re.split(r'[\s,; /] +', text)
# words = [word for word in words if word]
```

re.sub(pattern, new, text, count=0, flags=0)

- regex.sub(pattern, new, count=0, flags=0)
- 查找模式pattern，将匹配的内容替换为new，返回替换后的字符串
- text:
 - 字符串
 - 字符串，可以包含\1 \2
 - 函数对象func，替代为func(match)
- count为最大替换次数，缺省为全部替换
- 注意方法名不是replace

```
text = input('...')
text = re.sub(r'[\s,; /] +', ' ', text)
words = text.split( )
```

- 掌握使用search/finditer方法获得匹配内容的方法
- 了解match, fullmatch方法
- 掌握使用findall匹配时正则表达式出现分组符如何处理
- 掌握使用sub方法进行替代的方法
- 掌握使用split方法进行分割的方法